

Visualisation — Matplotlib & Seaborn

By the end of today you will be able to create professional charts that communicate data insights clearly. You will know which chart to use for which situation, how to make your plots clean and readable, and how to use both Matplotlib (the foundation) and Seaborn (the beautiful shortcut) confidently. Good visualisation is what turns a data analysis into a story.

Dataset for today: Use the cleaned Titanic CSV from Day 8. Mac users: activate venv first with `source venv/bin/activate`

SECTION 01 Install Matplotlib and Seaborn

Both libraries need to be installed once. After that, just import them at the top of any file.

✂ Install both libraries

Mac (with venv activated):

```
pip install matplotlib seaborn
```

Windows:

```
pip install matplotlib seaborn
```

Linux:

```
pip3 install matplotlib seaborn
```

Verify both installed correctly:

```
python3 -c "import matplotlib; import seaborn; print('Both ready!')"
```

Standard imports you will write at the top of EVERY visualisation file:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

The aliases plt and sns are universal standards. Always use them.

SECTION 02 Matplotlib vs Seaborn — What is the Difference?

Students often wonder: if we have Seaborn, why learn Matplotlib? The answer is that Seaborn is built ON TOP of Matplotlib. Understanding both makes you a complete visualisation professional.

	Matplotlib	Seaborn
What it is	The foundation. Low-level plotting library. Maximum control.	Built on top of Matplotlib. High-level shortcuts for statistical charts.
Effort	More lines of code for the same chart.	Fewer lines, beautiful defaults out of the box.
Best for	Custom charts, fine-tuned layouts, non-standard visuals.	Statistical plots: distributions, relationships, categorical comparisons.
Default style	Plain, basic, needs manual styling.	Polished, publication-quality with minimal effort.
Works with	NumPy arrays, Python lists.	Pandas DataFrames natively. Understands column names.

The professional workflow:

Use Seaborn for quick, beautiful statistical charts during EDA.
Use Matplotlib to customise, combine, annotate, and fine-tune those charts.
They work together seamlessly — you can create a Seaborn chart and then use Matplotlib commands to add titles, labels, and formatting on top of it.

SECTION 03 Matplotlib Fundamentals — The Building Blocks

Before making any chart, you need to understand the anatomy of a Matplotlib figure. Every chart is built from the same parts.

Anatomy of a Matplotlib chart:

Figure — The entire canvas. Think of it as a blank page.
Axes — One chart within the figure. A figure can have multiple axes (subplots).
X-axis — The horizontal axis. Usually the independent variable.
Y-axis — The vertical axis. Usually the dependent variable.
Title — Headline of the chart. Says what the chart is about.
Labels — Names for the X and Y axes. Always include units.
Legend — Explains what each colour or line means.

Ticks — The marks on the axis with their values.

Your first Matplotlib chart — the Hello World of visualisation

Every Matplotlib chart follows the same three-line pattern: create the plot, add labels, show it. Run this code and a window will pop up with your chart.

```
import matplotlib.pyplot as plt
import numpy as np

# Data
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [45000, 52000, 48000, 61000, 58000, 70000]

# Create the figure and axes
plt.figure(figsize=(10, 6)) # width=10 inches, height=6 inches

# Plot the data
plt.plot(months, sales, color="steelblue", linewidth=2,
         marker="o", markersize=8, label="Monthly Sales")

# Add labels and title
plt.title("Monthly Sales Performance 2024", fontsize=16, fontweight="bold",
pad=15)
plt.xlabel("Month", fontsize=13)
plt.ylabel("Sales (Rs.)", fontsize=13)

# Add a grid for readability
plt.grid(True, linestyle="--", alpha=0.7)

# Add the legend
plt.legend(fontsize=11)

# Save the chart as an image file
plt.savefig("sales_chart.png", dpi=150, bbox_inches="tight")

# Show the chart
plt.show()
```

Output / Chart:

A line chart appears showing monthly sales rising from 45,000 in January to 70,000 in June, with blue circles marking each data point, a grid in the background, and a clean title at the top.

File 'sales_chart.png' is also saved in your working folder.

Important for Mac users:

If `plt.show()` opens a window but it freezes VS Code, use this instead:

```
plt.savefig('chart.png') # save to file
plt.close()             # close the plot
```

Then open the PNG file in Finder to view it. This always works.
Alternatively, use Google Colab where charts display inline automatically.

The essential styling parameters

Parameter	Values	What it controls
color	'red', '#FF5733', 'steelblue'	Colour of the line or bars
linewidth / lw	1, 2, 3 (number)	Thickness of lines
linestyle / ls	'-', '--', ':', '-.'	Solid, dashed, dotted, dash-dot
marker	'o', 's', '^', '*', '+'	Shape of data point markers
markersize / ms	4, 6, 8 (number)	Size of markers
alpha	0.0 to 1.0	Transparency (0=invisible, 1=solid)
figsize	(width, height) in inches	Size of the entire figure
fontsize	10, 12, 14 (number)	Text size for titles and labels
label	'any string'	Text shown in the legend for this series

SECTION 04 Line Chart — Showing Trends Over Time

□ Line Chart

What this chart shows: Shows how a value changes over time or a continuous sequence. Perfect for trends, patterns, and comparisons of multiple series over the same time period.

What to look for: Direction of movement (going up or down?), rate of change (steep or gradual?), seasonal patterns, and where two lines cross each other.

Use a line chart when: your X-axis is time or an ordered sequence, and you want to show how values change.

```
import matplotlib.pyplot as plt
import numpy as np

# Monthly revenue for three products over 12 months
months = range(1, 13)
```

```

product_a = [45, 52, 48, 61, 58, 70, 65, 72, 68, 80, 85, 92]
product_b = [30, 35, 40, 38, 45, 50, 48, 55, 60, 58, 62, 70]
product_c = [20, 22, 25, 30, 28, 32, 35, 40, 38, 45, 50, 55]
month_names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
               "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

plt.figure(figsize=(12, 6))

# Plot three lines
plt.plot(month_names, product_a, color="#2196F3", lw=2.5,
         marker="o", ms=7, label="Product A")
plt.plot(month_names, product_b, color="#4CAF50", lw=2.5,
         marker="s", ms=7, label="Product B")
plt.plot(month_names, product_c, color="#FF9800", lw=2.5,
         marker="^", ms=7, label="Product C")

# Styling
plt.title("Monthly Revenue by Product (2024)", fontsize=16,
         fontweight="bold")
plt.xlabel("Month", fontsize=13)
plt.ylabel("Revenue (Rs. thousands)", fontsize=13)
plt.legend(fontsize=11)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout() # Auto-adjusts spacing so nothing gets cut off
plt.savefig("line_chart.png", dpi=150, bbox_inches="tight")
plt.show()

```

Output / Chart:

Three rising lines plotted over 12 months.
Product A (blue circles) is highest, growing from 45 to 92.
Product B (green squares) is in the middle, growing from 30 to 70.
Product C (orange triangles) is lowest, growing from 20 to 55.
All three show upward trends confirming business growth.

SECTION 05 Bar Chart — Comparing Categories

□ Bar Chart

What this chart shows: Compares values across different categories. Each bar's height represents the value for that category. Perfect for comparing totals, counts, or averages across groups.

What to look for: Which bar is tallest (highest value)? How large is the gap between bars? Are any values surprisingly low or high compared to others?

Use a bar chart when: you are comparing discrete categories (cities, products, classes, genders) and the order does not inherently matter.

```

import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("titanic_cleaned.csv")

# Bar chart: number of passengers per class
class_counts = df["Pclass"].value_counts().sort_index()

plt.figure(figsize=(8, 6))

bars = plt.bar(
    x      = ["First", "Second", "Third"],
    height = class_counts.values,
    color  = ["#1F5C8B", "#2E86C1", "#85C1E9"],
    edgecolor = "white",
    linewidth = 1.5,
    width   = 0.6
)

# Add value labels on top of each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        x      = bar.get_x() + bar.get_width() / 2,
        y      = height + 5,
        s      = str(int(height)),
        ha     = "center",
        va     = "bottom",
        fontsize = 12,
        fontweight = "bold"
    )

plt.title("Passengers by Class – Titanic", fontsize=16, fontweight="bold")
plt.xlabel("Passenger Class", fontsize=13)
plt.ylabel("Number of Passengers", fontsize=13)
plt.ylim(0, 560) # Give space for value labels
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("bar_chart.png", dpi=150, bbox_inches="tight")
plt.show()

```

Output / Chart:

Three bars showing: First class=216, Second class=184, Third class=491. Third class is by far the tallest bar (more than double First class), confirming that the Titanic carried mostly third-class passengers. Value labels float above each bar for immediate readability.

Horizontal bar chart — when labels are long

When your category names are long, use a horizontal bar chart so the labels do not overlap. Use `plt.barh()` instead of `plt.bar()`.

```

import matplotlib.pyplot as plt

cities    = ["Mumbai", "Bengaluru", "Delhi", "Hyderabad", "Chennai",
"Pune"]
startups  = [2800, 2100, 1900, 1200, 900, 750]

# Sort for better readability
sorted_pairs = sorted(zip(startups, cities))
startups_sorted, cities_sorted = zip(*sorted_pairs)

plt.figure(figsize=(10, 6))
bars = plt.barh(cities_sorted, startups_sorted, color="#2E86C1",
                edgcolor="white", height=0.6)

# Add value labels
for bar in bars:
    width = bar.get_width()
    plt.text(width + 30, bar.get_y() + bar.get_height()/2,
            f"{int(width):,}", va="center", fontsize=11)

plt.title("Tech Startups by City – India 2024", fontsize=15,
fontweight="bold")
plt.xlabel("Number of Startups", fontsize=12)
plt.grid(axis="x", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("horizontal_bar.png", dpi=150, bbox_inches="tight")
plt.show()

```

SECTION 06 Histogram — Understanding Distributions

□ Histogram

What this chart shows: Shows how values are distributed across a range. Divides data into equal-width bins and shows how many values fall in each bin. Reveals whether data is symmetric, skewed, or has outliers.

What to look for: Is the shape symmetric (normal distribution) or skewed left/right? Where is the peak (most common value)? Are there gaps or unusual spikes? Are there outliers far from the main group?

Use a histogram when: you want to understand the distribution of a single numeric variable. This is one of the very first things you do in EDA.

```

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = pd.read_csv("titanic_cleaned.csv")

```

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left chart: Age distribution
axes[0].hist(df["Age"].dropna(), bins=30, color="#2196F3",
             edgecolor="white", linewidth=0.8)
axes[0].axvline(df["Age"].mean(), color="red", ls="--",
               lw=2, label=f"Mean: {df['Age'].mean():.1f}")
axes[0].axvline(df["Age"].median(), color="orange", ls="--",
               lw=2, label=f"Median: {df['Age'].median():.1f}")
axes[0].set_title("Age Distribution", fontsize=14, fontweight="bold")
axes[0].set_xlabel("Age", fontsize=12)
axes[0].set_ylabel("Count", fontsize=12)
axes[0].legend(fontsize=11)
axes[0].grid(axis="y", linestyle="--", alpha=0.6)

# Right chart: Fare distribution
axes[1].hist(df["Fare"], bins=40, color="#4CAF50",
             edgecolor="white", linewidth=0.8)
axes[1].set_title("Fare Distribution", fontsize=14, fontweight="bold")
axes[1].set_xlabel("Fare (GBP)", fontsize=12)
axes[1].set_ylabel("Count", fontsize=12)
axes[1].grid(axis="y", linestyle="--", alpha=0.6)

plt.suptitle("Titanic: Numeric Variable Distributions",
             fontsize=16, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("histograms.png", dpi=150, bbox_inches="tight")
plt.show()

```

Output / Chart:

Left chart: Age distribution peaks around 20-35, slightly right-skewed. Mean (red dashed line) at ~29.4, Median (orange dashed line) at ~28. They are close, confirming slight skew.

Right chart: Fare distribution is heavily right-skewed. Most passengers paid under 50 GBP but a few paid over 500 GBP (outliers). This is the classic shape of income/price data.

Reading a histogram — three shapes you must know:

Normal (bell curve) — Mean = Median. Symmetric. Most data near centre.

Example: Heights of adult men in a city.

Right-skewed — Long tail on the RIGHT. Mean > Median.

Most values are low but a few are very high.

Example: Salaries, house prices, Fare in Titanic.

Left-skewed — Long tail on the LEFT. Mean < Median.

Most values are high but a few are very low.
Example: Age at retirement, exam scores (hard exam).

SECTION 07 Scatter Plot — Finding Relationships Between Variables

□ Scatter Plot

What this chart shows: Shows the relationship between two numeric variables. Each point represents one data row. The position of the point shows its value on both axes.

What to look for: Is there a pattern? Do points go up-right together (positive correlation)? Down-right (negative)? Is there no pattern (no correlation)? Are there clusters? Are there outliers far from the main group?

Use a scatter plot when: you want to see if two numeric variables are related to each other. This is critical before building any regression model.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

df = pd.read_csv("titanic_cleaned.csv")

plt.figure(figsize=(10, 6))

# Scatter: Age vs Fare, coloured by survival
survivors = df[df["Survived"] == 1]
non_survivors = df[df["Survived"] == 0]

plt.scatter(non_survivors["Age"], non_survivors["Fare"],
            color="#E74C3C", alpha=0.5, s=40,
            label="Did Not Survive", edgecolors="white", lw=0.5)

plt.scatter(survivors["Age"], survivors["Fare"],
            color="#2ECC71", alpha=0.5, s=40,
            label="Survived", edgecolors="white", lw=0.5)

plt.title("Age vs Fare - Coloured by Survival", fontsize=15,
          fontweight="bold")
plt.xlabel("Age", fontsize=13)
plt.ylabel("Fare (GBP)", fontsize=13)
plt.legend(fontsize=11)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("scatter_plot.png", dpi=150, bbox_inches="tight")
plt.show()
```

Output / Chart:

A scatter plot with green dots (survived) and red dots (did not survive). Observation: passengers who paid higher fares (top of chart) are more likely to be green (survived). Younger passengers with low fares are more red. This visually confirms what groupby told us: class (correlated with fare) strongly influenced survival.

SECTION 08 Pie Chart — Showing Proportions

□ Pie Chart

What this chart shows: Shows the proportion of each category as a slice of a whole circle. Best for showing how a total is divided into parts. Works well with 3-5 categories.

What to look for: Which slice is largest? What percentage does each part represent? Are any slices surprisingly small or large?

Use a pie chart when: you have fewer than 6 categories and want to show how a whole is divided. Avoid for more than 6 slices — it becomes unreadable.

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("titanic_cleaned.csv")

# Embarked port distribution
port_counts = df["Embarked"].value_counts()
port_labels = {"S": "Southampton", "C": "Cherbourg", "Q": "Queenstown"}
labels      = [port_labels.get(p, p) for p in port_counts.index]

plt.figure(figsize=(8, 8))
wedges, texts, autotexts = plt.pie(
    port_counts.values,
    labels      = labels,
    autopct    = "%1.1f%%",      # Show percentage on each slice
    startangle= 90,              # Start from top
    colors     = ["#2196F3", "#4CAF50", "#FF9800"],
    explode    = [0.05, 0.05, 0.05], # Slight separation between slices
    shadow     = True
)

# Style the percentage text
for autotext in autotexts:
    autotext.set_fontsize(13)
    autotext.set_fontweight("bold")

plt.title("Port of Embarkation — Titanic Passengers",
```

```
        fontsize=15, fontweight="bold", pad=20)
plt.tight_layout()
plt.savefig("pie_chart.png", dpi=150, bbox_inches="tight")
plt.show()
```

Output / Chart:

Three slices: Southampton 72.4% (largest, dark blue), Cherbourg 18.9% (medium, green), Queenstown 8.7% (smallest, orange). Most passengers boarded at Southampton, England.

SECTION 09 Seaborn — Beautiful Statistical Charts in Fewer Lines

Seaborn produces publication-quality statistical charts with much less code than raw Matplotlib. It works directly with Pandas DataFrames and understands column names, which makes the code very readable. This is what you will use most in EDA.

Standard Seaborn setup — always add these two lines at the top of your file:

```
import seaborn as sns
sns.set_theme(style='whitegrid') # Sets a clean white background with grid
```

style options: 'whitegrid', 'darkgrid', 'white', 'dark', 'ticks'
'whitegrid' is the most professional for data science presentations.

Box Plot — Five-Number Summary in One Chart

□ Box Plot (Box and Whisker)

What this chart shows: Shows the distribution of a numeric variable using five numbers: minimum, Q1 (25th percentile), median, Q3 (75th percentile), and maximum. Outliers are shown as individual dots.

What to look for: The box covers the middle 50% of data (IQR). The line inside the box is the median. Whiskers extend to the min and max within 1.5*IQR. Any dots beyond the whiskers are outliers.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="whitegrid")
df = pd.read_csv("titanic_cleaned.csv")

plt.figure(figsize=(10, 6))
```

```
# Box plot: Fare distribution by passenger class
sns.boxplot(
    data    = df,
    x       = "Pclass",
    y       = "Fare",
    palette = "Blues",
    order   = [1, 2, 3]
)

plt.title("Fare Distribution by Passenger Class",
          fontsize=15, fontweight="bold")
plt.xlabel("Passenger Class", fontsize=13)
plt.ylabel("Fare (GBP)", fontsize=13)
plt.tight_layout()
plt.savefig("boxplot.png", dpi=150, bbox_inches="tight")
plt.show()
```

Output / Chart:

Three box plots side by side.
 First class: box is high (median ~60 GBP) with many high outliers up to 512 GBP.
 Second class: box is in the middle (median ~15 GBP), tighter spread.
 Third class: box is low (median ~8 GBP), very compact.
 The boxes visually confirm the massive fare inequality between classes.

Violin Plot — Distribution Shape + Box Plot Combined

A violin plot is like a box plot but richer. It shows the full distribution shape on both sides. Where the violin is wider, more data points exist there.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="whitegrid")
df = pd.read_csv("titanic_cleaned.csv")

plt.figure(figsize=(10, 6))

sns.violinplot(
    data    = df,
    x       = "Pclass",
    y       = "Age",
    palette = "muted",
    inner   = "box"      # Show box plot inside the violin
)

plt.title("Age Distribution by Passenger Class",
          fontsize=15, fontweight="bold")
```

```
plt.xlabel("Passenger Class", fontsize=13)
plt.ylabel("Age (years)", fontsize=13)
plt.tight_layout()
plt.savefig("violin_plot.png", dpi=150, bbox_inches="tight")
plt.show()
```

Count Plot — Categorical Frequency

A count plot is a bar chart that automatically counts and plots how many rows belong to each category. Perfect for categorical columns.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="whitegrid")
df = pd.read_csv("titanic_cleaned.csv")

plt.figure(figsize=(10, 6))

# Count of survivors vs non-survivors, split by gender
sns.countplot(
    data    = df,
    x       = "Survived",
    hue     = "Sex",
    palette = {"male": "#2196F3", "female": "#E91E63"}
)

# Replace 0 and 1 with readable labels
plt.xticks([0, 1], ["Did Not Survive", "Survived"], fontsize=12)
plt.title("Survival Count by Gender", fontsize=15, fontweight="bold")
plt.xlabel("", fontsize=13)
plt.ylabel("Number of Passengers", fontsize=13)
plt.legend(title="Gender", fontsize=11)
plt.tight_layout()
plt.savefig("count_plot.png", dpi=150, bbox_inches="tight")
plt.show()
```

Output / Chart:

Grouped bars showing: far more males did not survive than females.
 Among survivors, females outnumber males by a wide margin.
 This single chart tells the 'women and children first' story instantly.

Heatmap — Correlation Matrix Visualised

A heatmap uses colour intensity to show the strength of values in a matrix. The most common use in data science is visualising the correlation matrix to spot which features are related to each other and to the target variable.

```

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="white")
df = pd.read_csv("titanic_cleaned.csv")

# Compute correlation matrix on numeric columns only
numeric_df = df.select_dtypes(include="number")
corr_matrix = numeric_df.corr()

plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
    annot      = True,          # Show numbers in each cell
    fmt        = ".2f",        # Round to 2 decimal places
    cmap       = "coolwarm",    # Blue=negative, Red=positive
    center     = 0,            # Centre colour scale at 0
    square     = True,          # Make cells square
    linewidths = 0.5,
    cbar_kws   = {"shrink": 0.8}
)

plt.title("Correlation Matrix - Titanic Dataset",
          fontsize=15, fontweight="bold", pad=15)
plt.xticks(rotation=45, ha="right", fontsize=11)
plt.yticks(rotation=0,  fontsize=11)
plt.tight_layout()
plt.savefig("heatmap.png", dpi=150, bbox_inches="tight")
plt.show()

```

Output / Chart:

A colour-coded grid. Each cell shows the correlation between two variables. Deep red = strong positive correlation (close to +1). Deep blue = strong negative correlation (close to -1). White/pale = near zero correlation. Diagonal is always 1.0 (each variable perfectly correlates with itself).

Key insight: Fare and Pclass show strong negative correlation (-0.55), meaning higher class = lower class number = higher fare. Makes sense.

The correlation heatmap is one of the most important charts in all of data science EDA. In one glance it tells you: which features correlate with the target (Survived)? Which features are too similar to each other (multicollinearity)? This guides your feature selection before building any ML model.

Pair Plot — Every Variable Against Every Other

A pair plot (also called a scatter matrix) creates a grid of scatter plots for every combination of numeric variables. The diagonal shows each variable's own distribution. One command, massive insight.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

sns.set_theme(style="whitegrid")
df = pd.read_csv("titanic_cleaned.csv")

# Select relevant numeric columns only (pair plot gets busy with too many)
cols = ["Age", "Fare", "SibSp", "Parch", "Survived"]
df_pair = df[cols].dropna()

# Create pair plot coloured by survival status
pair_plot = sns.pairplot(
    data = df_pair,
    hue = "Survived",
    palette = {0: "#E74C3C", 1: "#2ECC71"},
    diag_kind= "kde",          # Smooth density curve on diagonal
    plot_kws = {"alpha": 0.5, "s": 30}
)

pair_plot.figure.suptitle("Pair Plot - Titanic Numeric Features",
                          y=1.02, fontsize=14, fontweight="bold")
plt.savefig("pair_plot.png", dpi=120, bbox_inches="tight")
plt.show()
```

Output / Chart:

A grid of charts. For 5 columns, you get a 5x5 grid = 25 mini-charts. Green points = survived, red points = did not survive. At a glance you can see which variable combinations separate the two groups. This single chart replaces 20 individual scatter plots.

SECTION 10 Subplots — Multiple Charts in One Figure

Subplots let you display multiple charts side by side or in a grid within one figure. This is essential for EDA reports and presentations where you want to compare several charts together.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

sns.set_theme(style="whitegrid")
```

```

df = pd.read_csv("titanic_cleaned.csv")

# Create a 2x2 grid of subplots
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(14, 10))
fig.suptitle("Titanic EDA Dashboard", fontsize=18, fontweight="bold",
y=1.01)

# Chart 1 (top-left): Passenger class distribution
df["Pclass"].value_counts().sort_index().plot(
    kind="bar", ax=axes[0,0], color=["#1F5C8B", "#2E86C1", "#85C1E9"],
    edgecolor="white")
axes[0,0].set_title("Passengers by Class", fontweight="bold")
axes[0,0].set_xlabel("Class"); axes[0,0].set_ylabel("Count")
axes[0,0].tick_params(axis="x", rotation=0)

# Chart 2 (top-right): Age histogram
axes[0,1].hist(df["Age"].dropna(), bins=25, color="#4CAF50",
    edgecolor="white")
axes[0,1].axvline(df["Age"].median(), color="red", ls="--",
    label=f'Median: {df["Age"].median():.0f}')
axes[0,1].set_title("Age Distribution", fontweight="bold")
axes[0,1].set_xlabel("Age"); axes[0,1].set_ylabel("Count")
axes[0,1].legend()

# Chart 3 (bottom-left): Survival by gender
survival_gender = df.groupby("Sex")["Survived"].mean() * 100
survival_gender.plot(kind="bar", ax=axes[1,0],
    color=["#E91E63", "#2196F3"], edgecolor="white")
axes[1,0].set_title("Survival Rate by Gender (%)", fontweight="bold")
axes[1,0].set_xlabel("Gender"); axes[1,0].set_ylabel("Survival Rate (%)")
axes[1,0].tick_params(axis="x", rotation=0)
axes[1,0].set_ylim(0, 100)

# Chart 4 (bottom-right): Fare by class box plot
sns.boxplot(data=df, x="Pclass", y="Fare", palette="Blues",
    ax=axes[1,1], order=[1,2,3])
axes[1,1].set_title("Fare by Class", fontweight="bold")
axes[1,1].set_xlabel("Class"); axes[1,1].set_ylabel("Fare (GBP)")

plt.tight_layout()
plt.savefig("eda_dashboard.png", dpi=150, bbox_inches="tight")
plt.show()

```

Output / Chart:

A 2x2 grid of four charts in one figure:
 Top-left : Bar chart showing most passengers are in third class.
 Top-right : Age histogram peaking around 20-35 with median line.
 Bottom-left: Women had ~74% survival rate, men only ~19%.
 Bottom-right: First class fares are much higher with many outliers.

This is a complete EDA dashboard saved as a single PNG file.
 This is exactly what you include in your GitHub portfolio.

SECTION 11 Which Chart to Use — The Decision Guide

The most important skill in visualisation is not how to code the chart. It is knowing **WHICH** chart to use for **WHICH** question. Use this table every time you need to make a chart.

Your question	Best chart	Why
How does a value change over time?	Line chart	Shows trends and direction clearly
How do categories compare in value?	Bar chart	Height difference is easy to judge
What is the distribution of one variable?	Histogram	Shows shape, skew, outliers
Is there a relationship between two numeric variables?	Scatter plot	Shows direction and strength of relationship
What are the proportions of categories?	Pie chart	Whole = 100%, slices = parts
What is the spread and outliers of a variable?	Box plot	Shows median, IQR, and outliers
Full distribution shape for categories?	Violin plot	Richer than box plot, shows density
How many items in each category?	Count plot (Seaborn)	Auto-counts and plots frequency
How correlated are all my variables?	Heatmap	Colour shows correlation strength
Relationships between all variable pairs?	Pair plot	One command, 25 charts at once

SECTION 12 Interview Prep — Questions to Answer After Today

Visualisation questions come up in both technical and non-technical interviews. The ability to explain charts to non-technical stakeholders is as valuable as coding them.

Q — What is the difference between Matplotlib and Seaborn?

Matplotlib is the foundational plotting library in Python. It gives you complete control over every element of a chart but requires more lines of code and manual styling. Seaborn is built on top of Matplotlib and provides a higher-level interface specifically designed for statistical visualisation. It works natively with Pandas DataFrames, understands column names, and produces beautiful charts with minimal code and sensible defaults. In practice, most data scientists use Seaborn for

quick EDA charts and then use Matplotlib commands to customise, annotate, or combine multiple plots. They complement each other rather than replace each other.

Q — When would you use a histogram vs a box plot?

Both show the distribution of a single numeric variable but they emphasise different aspects. A histogram shows the full shape of the distribution — whether it is symmetric or skewed, where the peak is, whether there are multiple humps. It is best for understanding the overall pattern. A box plot shows a five-number summary: minimum, Q1, median, Q3, maximum, and outliers as dots. It is best for comparing distributions across categories and for spotting outliers quickly. In practice I use histograms first to understand distribution shape, then box plots when I want to compare the same variable across groups, like Age across different passenger classes.

Q — What is a correlation heatmap and why is it important in EDA?

A correlation heatmap is a colour-coded grid where each cell shows the Pearson correlation coefficient between two variables. Colours closer to red indicate strong positive correlation (both variables increase together) and colours closer to blue indicate strong negative correlation (one increases as the other decreases). It is important in EDA for two reasons. First, it shows which features are most correlated with the target variable, helping prioritise which features to focus on. Second, it reveals multicollinearity — when two input features are too correlated with each other, which can cause problems in linear models. In the Titanic dataset, Pclass and Fare show a strong negative correlation of about -0.55, which makes sense because higher class means lower class number but higher price.

Q — How do you make a chart readable for a non-technical audience?

I follow five rules. First, always add a clear, descriptive title that states the main insight, not just what the chart shows. Instead of 'Survival by Gender' write 'Women Survived at 4x the Rate of Men.' Second, label all axes with units so nothing requires guessing. Third, add value labels on bars so the reader does not need to estimate from the axis. Fourth, use colour intentionally and consistently throughout a report. Fifth, add a brief text annotation or caption explaining the key takeaway. The goal is that someone who has never seen the dataset should understand the chart's message in under 10 seconds without explanation from me.

SECTION 13 Words to Know

Term	Definition
Matplotlib	The foundational Python plotting library. Maximum control, more code.
Seaborn	High-level statistical plotting built on Matplotlib. Beautiful defaults.
plt	Standard alias for matplotlib.pyplot. Always import as plt.
sns	Standard alias for seaborn. Always import as sns.
Figure	The entire canvas containing one or more charts.
Axes	A single chart within a figure. Has its own x-axis, y-axis, title.

<code>plt.figure()</code>	Create a new figure, usually with <code>figsize</code> parameter.
<code>plt.subplots()</code>	Create a figure with multiple subplot axes in a grid.
<code>plt.show()</code>	Display the chart. Call after all styling is complete.
<code>plt.savefig()</code>	Save chart to a file (PNG, PDF, SVG). Use before <code>plt.show()</code> .
<code>plt.tight_layout()</code>	Auto-adjust spacing so labels and titles do not get cut off.
Line chart	<code>plt.plot()</code> . Shows trends over time or ordered sequences.
Bar chart	<code>plt.bar()</code> or <code>plt.barh()</code> . Compares values across categories.
Histogram	<code>plt.hist()</code> . Shows distribution of one numeric variable.
Scatter plot	<code>plt.scatter()</code> . Shows relationship between two numeric variables.
Pie chart	<code>plt.pie()</code> . Shows proportions of a whole.
Box plot	<code>sns.boxplot()</code> . Five-number summary with outliers.
Violin plot	<code>sns.violinplot()</code> . Box plot + full distribution shape.
Count plot	<code>sns.countplot()</code> . Bar chart that auto-counts category frequency.
Heatmap	<code>sns.heatmap()</code> . Colour-coded matrix, used for correlation.
Pair plot	<code>sns.pairplot()</code> . Grid of scatter plots for all variable combinations.
<code>alpha</code>	Transparency setting. 0.0 = invisible, 1.0 = fully solid.
<code>hue</code>	Seaborn parameter. Colour data points by a categorical column.
<code>palette</code>	Seaborn parameter. The colour scheme to use for categories.

SECTION HW Homework — Submit Before Day 10

Create `day9_homework.py` using the cleaned Titanic dataset. Save every chart as a PNG file and push both the code and the PNG files to your GitHub repo. Every chart must have a title, labelled axes, and a grid.

Exercise 1 — Line Chart (Beginner)

Create a line chart showing a hypothetical stock price over 30 days. Generate the data using numpy: start at Rs. 1000 and add random daily changes (use `np.random.randn`). Plot the price with a red dashed horizontal line at the starting price to show profit/loss at a glance. Add title, labels, legend, and grid.

Exercise 2 — Bar and Horizontal Bar (Beginner-Intermediate)

Using the Titanic dataset: (1) Create a bar chart showing the average fare paid by each passenger class. Add value labels on each bar. (2) Create a horizontal bar chart showing survival rate (percentage) by embarkation port, sorted from highest to lowest. Add value labels.

Exercise 3 — Histogram Analysis (Intermediate)

Create a figure with two histograms side by side: one for Age, one for Fare from the Titanic dataset. On each histogram: add a vertical line for mean (red) and median (orange) with labels. Write a comment in your code explaining what the shape of each distribution tells you and why mean and median differ.

Exercise 4 — Seaborn Charts (Intermediate)

Using Seaborn on the Titanic dataset, create four charts in a 2x2 subplot grid: (1) Box plot of Age grouped by Pclass. (2) Violin plot of Fare grouped by Sex. (3) Count plot of Pclass grouped by Survived (use hue). (4) Heatmap of the correlation matrix. Save the entire grid as one PNG file called `seaborn_dashboard.png`.

Exercise 5 — Story Chart (Intermediate)

Create one chart that tells a complete story from the Titanic data. Choose any insight from your EDA this week. The chart must: have a title that states the insight (not just describes the axes), use colour intentionally to highlight the key finding, have labelled axes with units, and have a text annotation pointing to the most important part of the chart using `plt.annotate()`. Push to GitHub with a one-paragraph explanation in the README of what the chart reveals.

Stretch Challenge — Tier A students

Build a complete 6-chart Titanic EDA report as a single figure (3 rows x 2 columns). Each chart should answer a specific business question: (1) How many passengers per class? (2) Age distribution by survival status (overlapping histograms). (3) Survival rate by class AND gender (grouped bar). (4) Fare vs Age scatter coloured by survival. (5) Correlation heatmap. (6) Port of embarkation pie chart. Save as `titanic_eda_report.png` at high resolution (`dpi=200`). This chart goes directly in your portfolio README.